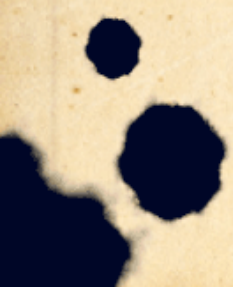




Data Engineering – DE-FT-2026

Final Capstone

Susmita Roy

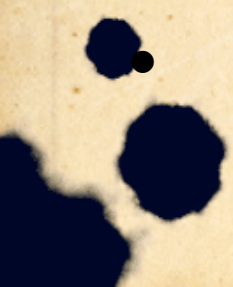


Banking System



Outline



- Basic knowledge what you need
 - Broad Idea about that
 - Phase wise (1-5)
 - SQL, Python, Airflow-dbt, Kafka-NIFI,
Together All
 - First some basic ideas
 - Then Tasks
- 

Phase 2

Telecom Customer Churn Dataset - Advanced Pandas Assessment

[4]:

	Data_Type	Null_Count	Null_Percentage	Distinct_Count	Duplicate_Count	Min_Value	Max_Value
customerID	object	0	0.0	7043	0	NaN	NaN
gender	object	0	0.0	2	7041	NaN	NaN
SeniorCitizen	int64	0	0.0	2	7041	0.00	1.00
Partner	object	0	0.0	2	7041	NaN	NaN
Dependents	object	0	0.0	2	7041	NaN	NaN
tenure	int64	0	0.0	73	6970	0.00	72.00
PhoneService	object	0	0.0	2	7041	NaN	NaN
MultipleLines	object	0	0.0	3	7040	NaN	NaN
InternetService	object	0	0.0	3	7040	NaN	NaN
OnlineSecurity	object	0	0.0	3	7040	NaN	NaN
OnlineBackup	object	0	0.0	3	7040	NaN	NaN
DeviceProtection	object	0	0.0	3	7040	NaN	NaN
TechSupport	object	0	0.0	3	7040	NaN	NaN
StreamingTV	object	0	0.0	3	7040	NaN	NaN
StreamingMovies	object	0	0.0	3	7040	NaN	NaN
Contract	object	0	0.0	3	7040	NaN	NaN
PaperlessBilling	object	0	0.0	2	7041	NaN	NaN
PaymentMethod	object	0	0.0	4	7039	NaN	NaN
MonthlyCharges	float64	0	0.0	1585	5458	18.25	118.75
TotalCharges	object	0	0.0	6531	512	NaN	NaN
Churn	object	0	0.0	2	7041	NaN	NaN

Telecom_customer_churn data:



```
[6]: import pandas as pd
import numpy as np

# Load the dataset
df = pd.read_csv("../Telecom_Customers_Churn.csv")

# Basic information
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print(df.columns)
df.head()
```

```
Rows: 7043
Columns: 21
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
      'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
      'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
      'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
      'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')
```

```
[6]:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	TechSu
--	------------	--------	---------------	---------	------------	--------	--------------	---------------	-----------------	----------------	-----	------------------	--------

0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	No
---	------------	--------	---	-----	----	---	----	------------------	-----	----	-----	----

1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	Yes
---	------------	------	---	----	----	----	-----	----	-----	-----	-----	-----

2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	No
---	------------	------	---	----	----	---	-----	----	-----	-----	-----	----

3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	Yes
---	------------	------	---	----	----	----	----	------------------	-----	-----	-----	-----

4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	No
---	------------	--------	---	----	----	---	-----	----	-------------	----	-----	----

5 rows × 21 columns

Telecom Customer Churn – Data Quality & Preparation

Python analysis of 7,043 customers and 21 variables

7,043
Customers

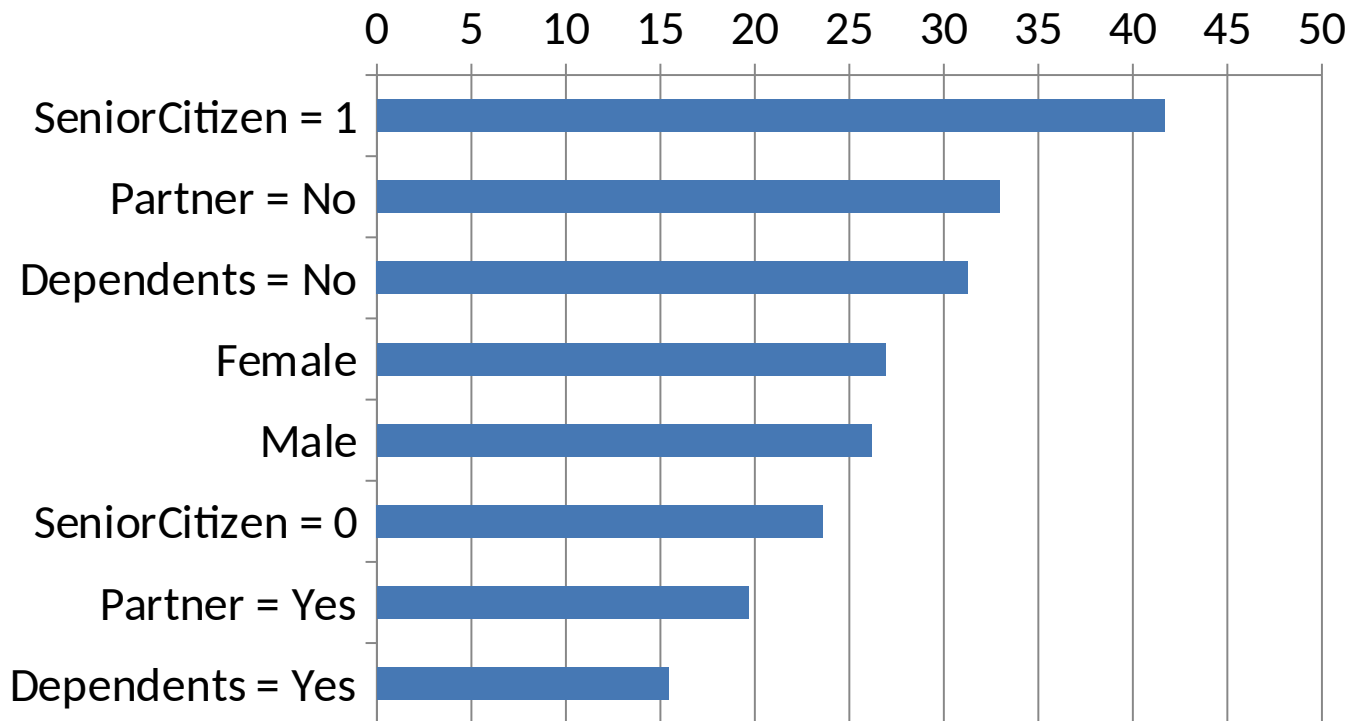
21
Columns

11
Invalid Total
Charge

5.41%
>10% charge
difference

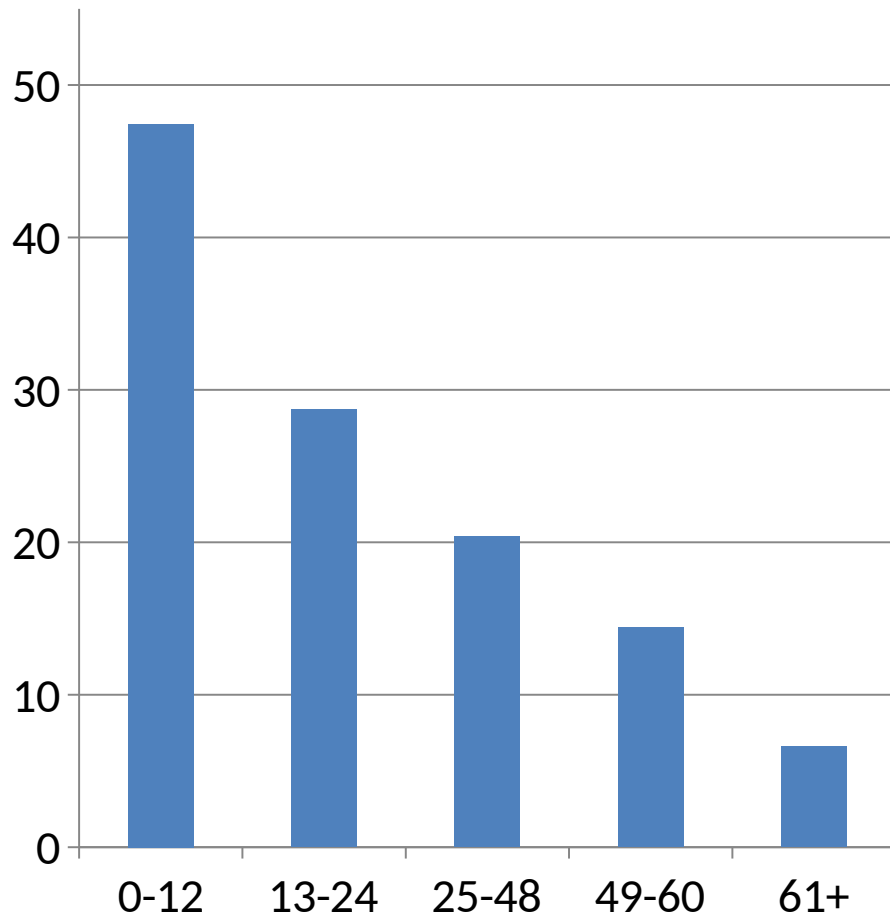
Who Is Most at Risk of Churning?

Churn rate by customer group



Tenure and Customer Profiles Drive Churn

Churn rate by tenure segment



Key Messages:

Churn falls strongly as customer tenure increases;

Senior Citizens are at most risk of Churning;

Key message: Early-tenure customers and month-to-month customers using electronic check with Fiber Internet show particularly high churn.

Data sets:

1. Bank Fraud dataset : Snow Flake

kafka_demo > CapstonePhase1.sql

ACCOUNTADMIN • COMPUTE_WH (X-Small) BANK_FRAUD PUBLIC

```
-- Select the BANK_FRAUD database
USE DATABASE BANK_FRAUD;
DESCRIBE TABLE bankfraud;
```

Results (2 minutes ago)

Table Chart Pivot

26 rows 75ms

	name	type	kind	null?	default	primary key	unique key	check	expression	comment
	ACCOUNT_A... 3.8%	NUMBER(38,0) 42.3%								
	ACCOUNT_B... 3.8%	VARCHAR(16777... 34.6%								
	+24 more	+3 more	COL... 100.0%	Y 100.0%	All values are null	N 100.0%	N 100.0%	All values are null	All values are null	All values are null
1	TRANSACTION_ID	VARCHAR(16777216)	COLUMN	Y	null	N	N	null	null	null
2	CUSTOMER_ID	VARCHAR(16777216)	COLUMN	Y	null	N	N	null	null	null
3	TRANSACTION_DATE	VARCHAR(16777216)	COLUMN	Y	null	N	N	null	null	null
4	TRANSACTION_TIME	TIME(9)	COLUMN	Y	null	N	N	null	null	null
5	HOURL_OF_DAY	NUMBER(38,0)	COLUMN	Y	null	N	N	null	null	null
6	IS_WEEKEND	NUMBER(38,0)	COLUMN	Y	null	N	N	null	null	null
7	IS_NIGHT_TRANSACTION	NUMBER(38,0)	COLUMN	Y	null	N	N	null	null	null
8	COUNTRY	VARCHAR(16777216)	COLUMN	Y	null	N	N	null	null	null
9	CITY	VARCHAR(16777216)	COLUMN	Y	null	N	N	null	null	null
10	MERCHANT_CATEGORY	VARCHAR(16777216)	COLUMN	Y	null	N	N	null	null	null
11	PAYMENT_METHOD	VARCHAR(16777216)	COLUMN	Y	null	N	N	null	null	null
12	DEVICE_TYPE	VARCHAR(16777216)	COLUMN	Y	null	N	N	null	null	null
13	CUSTOMER_AGE	NUMBER(38,0)	COLUMN	Y	null	N	N	null	null	null
14	CREDIT_SCORE	NUMBER(38,0)	COLUMN	Y	null	N	N	null	null	null
15	ACCOUNT_AGE_YEARS	NUMBER(38,1)	COLUMN	Y	null	N	N	null	null	null
16	ACCOUNT_BALANCE	NUMBER(38,2)	COLUMN	Y	null	N	N	null	null	null
17	TRANSACTION_AMOUNT	NUMBER(38,2)	COLUMN	Y	null	N	N	null	null	null

Module 1

Q1

Data exploration & profiling

Module 1: Data Exploration & Profiling

Q1 Calculate:

Total Transactions

Total Customers

Total Fraud Transactions

Fraud Percentage

```
-- Select the BANK_FRAUD database
USE DATABASE BANK_FRAUD;
DESCRIBE TABLE bankfraud;
```

Results (just now)

Table Chart Pivot

1 row 379ms

	# TOTAL_TRANSACTIONS	# TOTAL_CUSTOMERS	# TOTAL_FRAUD_TRANSACTIONS	# FRAUD_PERCENTAGE
1	1000000	198662	55255	5.53

Module 1

-----Q3 Find the top 10 cities generating the highest transaction value.

| Ctrl+I to generate

```
SELECT
  CITY,
  SUM(TRANSACTION_AMOUNT) AS total_transaction_value
FROM BANKFRAUD
GROUP BY CITY
ORDER BY total_transaction_value DESC
LIMIT 10;
```

Top 10 cities by transaction volume.

-----Q4. Generate a complete data profiling report showing:

-----Column Name

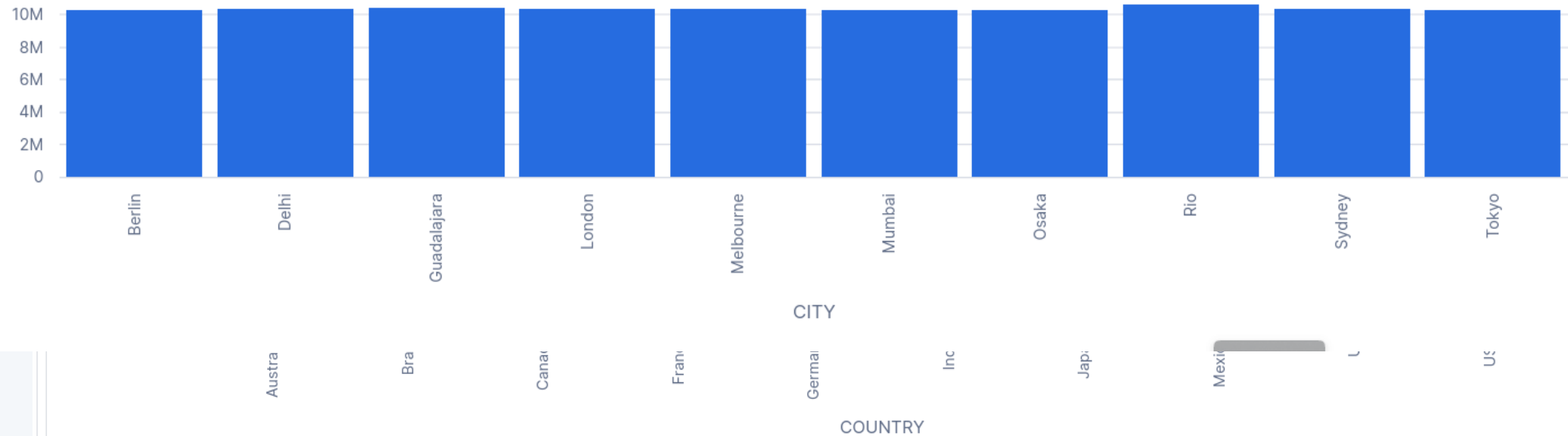
(1 minute ago)

Chart

Pivot

Series

■ TOTAL_TRANSACTION_VALUE



Customer behavior And Analysis

```
SELECT
  CASE
    WHEN customer_age BETWEEN 18 AND 25 THEN '18-25'
    WHEN customer_age BETWEEN 26 AND 35 THEN '26-35'
    WHEN customer_age BETWEEN 36 AND 50 THEN '36-50'
    WHEN customer_age BETWEEN 51 AND 65 THEN '51-65'
    WHEN customer_age > 65 THEN '65+'
    ELSE 'Unknown'
  END AS Customer_Age,
  ROUND(AVG(TRANSACTION_AMOUNT), 2) AS avg_transaction_amount
FROM BANKFRAUD
GROUP BY Customer_Age
```

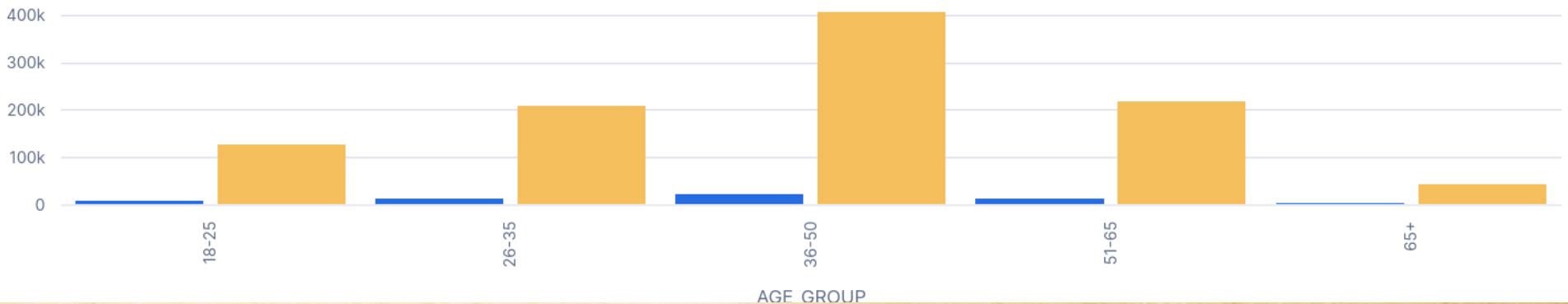
Results (3 minutes ago)

Table Chart Pivot



Series

FRAUD_TRANSACTIONS TOTAL_TRANSACTIONS



Fraud Pattern Analysis

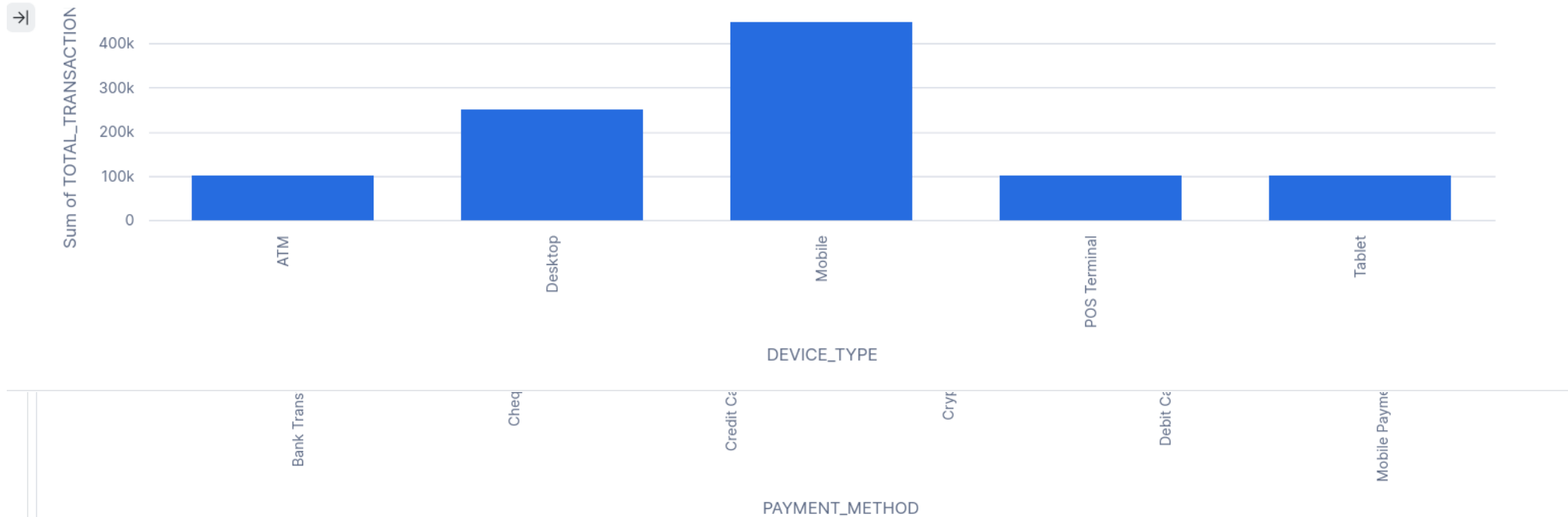
Fraud rate by Payment method :

Fraud rate by device type:

```
-- Q13: Fraud Rate by Device Type
SELECT
  DEVICE_TYPE,
  COUNT(*) AS total_transactions,
  SUM(CASE WHEN IS_FRAUD = 1 THEN 1 ELSE 0 END) AS fraud_transactions,
  ROUND((SUM(CASE WHEN IS_FRAUD = 1 THEN 1 ELSE 0 END) * 100.0) / COUNT(*), 2) AS fraud_rate_pct
FROM BANKFRAUD
GROUP BY DEVICE_TYPE
ORDER BY fraud_rate_pct DESC;
```

results (just now)

Table Chart Pivot



Fraud happens during

→ Key messages:

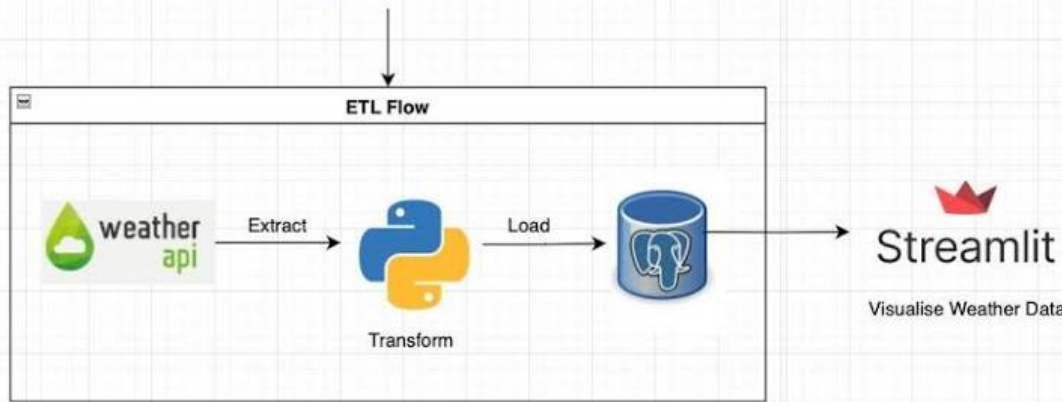
■ Weekday ■ Weekend

- The highest number of transactions and fraud transactions occur in the age group of 35 – 50 age group
- Most Fraud transactions occur through Credit Cards and mostly from Mobile devices
- Fraud Transactions occur more frequently on weekdays and during the day time



Phase 3

Airflow, dbt And Snow Flake



Open source Orchestration platform, used to schedule or monitor workflows through DAGS



Data building tool

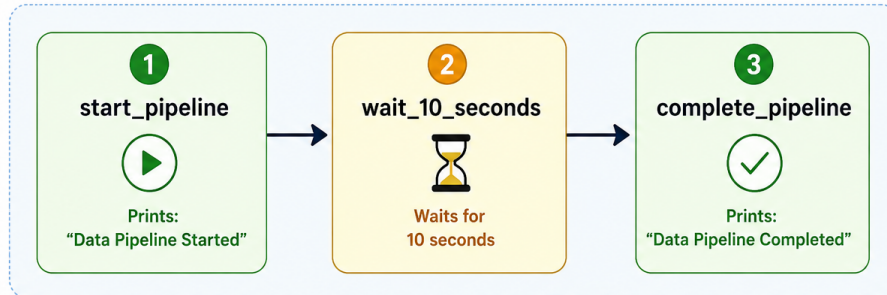


Cloud based Data Warehousing for storage, large transformation with SQL access

Task 1: Create a DAG that runs daily at 8:00 AM

Airflow DAG: first_pipeline

Schedule: Every day at 8:00 AM (0 8 * * *)



How it Works



In simple words:

The pipeline starts, prints a start message, waits for 10 seconds, then prints a completion message. All of this happens automatically every day at 8:00 AM.

Task execution order

`task1 >> task2 >> task3`

```
from datetime import datetime
import time
from airflow.sdk import DAG
from airflow.providers.standard.operators.python import
PythonOperator
```

```
def start_pipeline():
    print("Data Pipeline Started")
```

```
def wait_10_seconds():
    print("Waiting for 10 seconds...")
    time.sleep(10)
```

```
def complete_pipeline():
    print("Data Pipeline Completed")
```

```
with DAG(
    dag_id="first_pipeline",
    start_date=datetime(2026, 1, 1),
    schedule="0 8 * * *",
    catchup=False,
) as dag:
```

```
task1 = PythonOperator(
    task_id="start_pipeline",
    python_callable=start_pipeline,
)
```

```
task2 = PythonOperator(
    task_id="wait_10_seconds",
    python_callable=wait_10_seconds,
)
```

```
task3 = PythonOperator(
    task_id="complete_pipeline",
    python_callable=complete_pipeline,
)
```

Final View



Home

Dags

Assets

Browse

Admin

Dag

first_pipeline

00:00:14.154

00:00:07.077

start_pipeline

wait_10_seconds

complete_pipeline

✓

✓

✓

✓

✓

✓

+ Filter

 first_pipeline ☐

Schedule

0 8 ***

Latest Run

2026-08-31 15:32:43 

Next Run

2026-09-01 10:00:00

Max Active Runs

0 of 16

Owner

airflow

Tags

Latest Dag Version

v1

☆





Overview

Runs

Tasks

Calendar

Backfills

Audit Log

Code

Details

Q Search tasks

Select operators

Select trigger rules

Select retry values

Select mapped

3 Tasks

Task ID	Trigger Rule	Operator	Retries	Mapped
complete_pipeline	all_success	PythonOperator	0	false
start_pipeline	all_success	PythonOperator	0	false
wait_10_seconds	all_success	PythonOperator	0	false

Task 2: Snow Flake ETL Pipeline



Data: bank_fraud.csv,

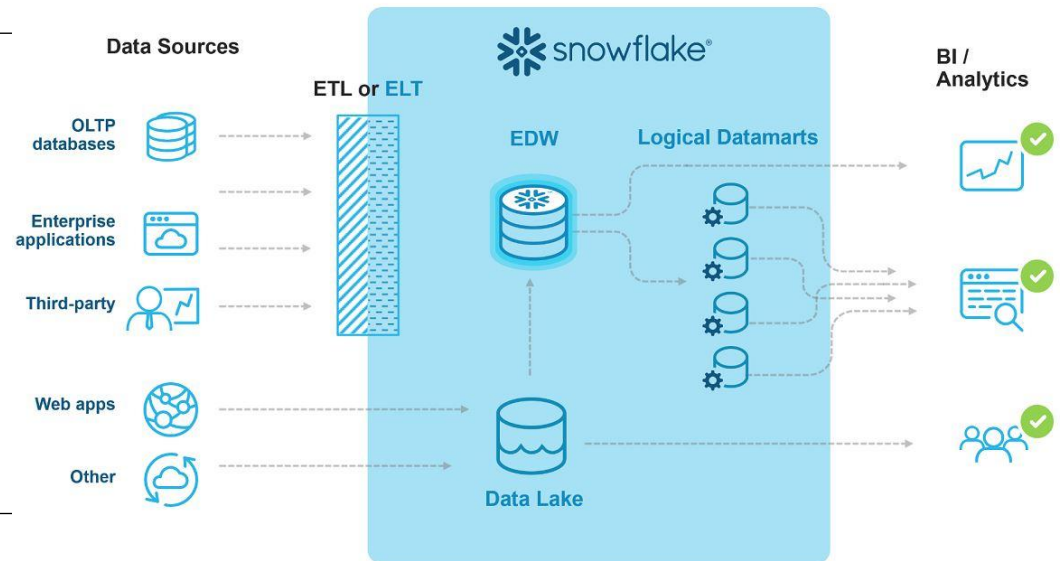
Goal: From raw banking transactions to analysis ready data.

Pipeline

Load with dbt using seeds feature----->Transform with dbt -----> mart-models with dbt -----> Analysis query with snow Flake



```
dbt_snowflake_CapsP3Q2Q3/  
├── dbt_project.yml  
├── seeds/  
│   └── bank_fraud.csv  
├── models/  
│   ├── staging/  
│   │   └── sources.yml  
│   ├── marts/  
│   │   ├── bank_fraud_analytics.sql  
│   │   ├── fraud_alerts.sql  
│   │   └── fraud_summary.sql  
└── README.md
```



Why?

Dbt based models are reusable

Open source platform

Easy to handle

Cloud based platform

Can handle Big data

Fast SQL analytics

Snow Flake Analytics

Resulting SnowFlake Table

Results (1 hour ago)

Resulting Snowflake Table

TableChartPivot

10 rows173ms

</

Tasks :

1. Generate row count Report

```
SELECT *
FROM DBT_ROW_COUNT_LOG
ORDER BY run_timestamp DESC;
```

	🕒 RUN_TIMESTAMP	<u>A</u> TABLE_NAME	# ROW_COUNT
1	2026-09-03 03:05:15.851	bank_fraud_analytics	1000000
2	2026-09-02 03:05:20.314	bank_fraud_analytics	1000000
3	2026-09-01 03:05:12.465	bank_fraud_analytics	1000000
4	2026-09-01 02:56:58.792	bank_fraud_analytics	1000000
5	2026-09-01 00:33:04.415	bank_fraud_analytics	1000000
6	2026-09-01 00:24:17.345	bank_fraud_analytics	1000000
7	2026-09-01 00:17:30.991	bank_fraud_analytics	1000000
8	2026-08-31 08:45:32.648	bank_fraud_analytics	1000000
9	2026-08-31 08:30:08.129	bank_fraud_analytics	1000000

Tasks

Building Fraud analysis Pipeline

Mart models:

Fraud detection rules

```
SELECT
    *,
    -- Rule 1: High Velocity (More than 3 transactions in
    a row where amount > 800)
    CASE
        WHEN TRANSACTION_AMOUNT > 800 AND
            COUNT(*) OVER (PARTITION BY
                CUSTOMER_ID ORDER BY tx_timestamp RANGE
                BETWEEN INTERVAL '1 HOUR' PRECEDING AND
                CURRENT ROW) >= 3
            THEN 1 ELSE 0
        END AS risk_rule_velocity,

    -- Rule 2: Suspicious Night Activities (High amount
    spent between 11 PM and 4 AM)
    CASE
        WHEN TRANSACTION_AMOUNT > 1500 AND
            EXTRACT(HOUR FROM tx_timestamp) IN (23, 0, 1, 2, 3,
            4)
            THEN 1 ELSE 0
        END AS risk_rule_night_spike,

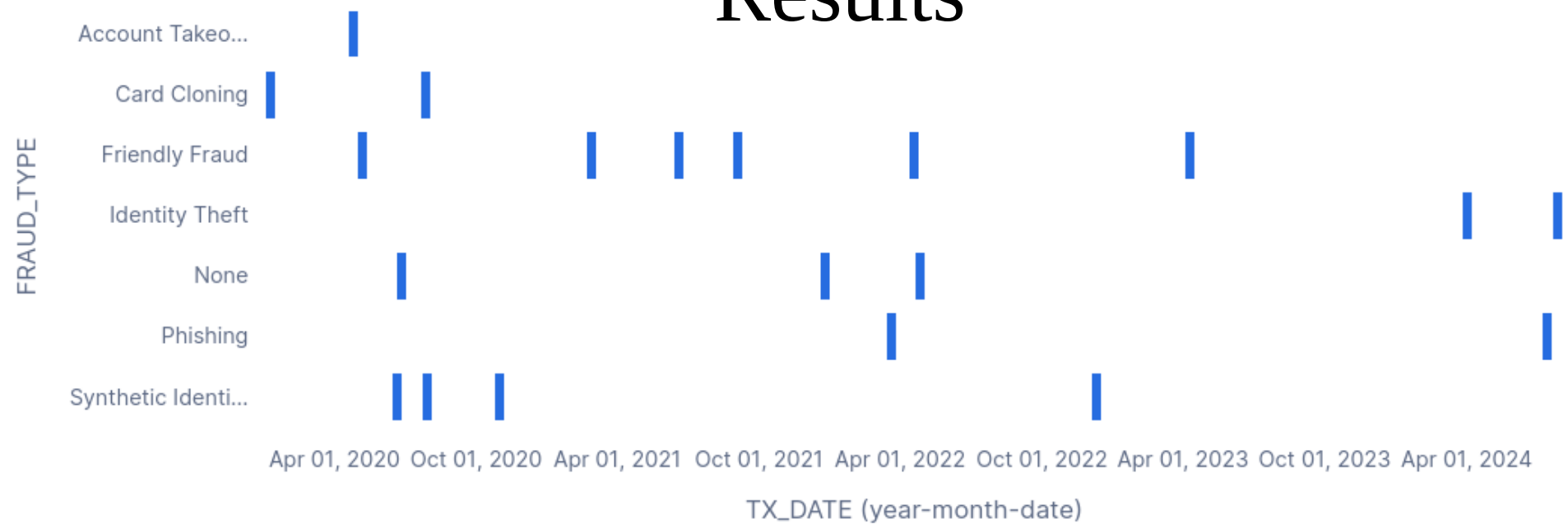
    -- Rule 3: Blacklisted/High-Risk Merchant Categories
    CASE
        WHEN MERCHANT_CATEGORY IN ('Online
        Gambling', 'Crypto Exchange', 'Luxury Goods') AND
            TRANSACTION_AMOUNT > 500
            THEN 1 ELSE 0
        END AS risk_rule_high_risk_merchant

FROM base_data
)
```

Flag suspicious transactions.

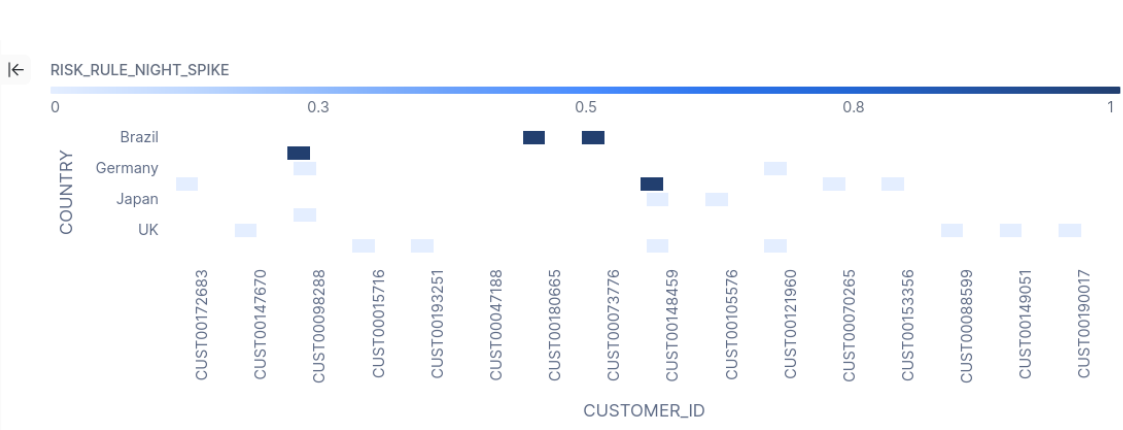
```
SELECT
    CUSTOMER_ID,
    tx_date,
    tx_time,
    tx_timestamp,
    CITY,
    COUNTRY,
    PAYMENT_METHOD,
    DEVICE_TYPE,
    TRANSACTION_AMOUNT,
    FRAUD_TYPE,
    risk_rule_velocity,
    risk_rule_night_spike,
    risk_rule_high_risk_merchant,
    -- If any individual rule fires, or if it was marked fraud historically, flag it as a risk
    alert
    CASE
        WHEN IS_FRAUD = 1 OR risk_rule_velocity = 1 OR risk_rule_night_spike = 1
        OR risk_rule_high_risk_merchant = 1
            THEN 1 ELSE 0
        END AS is_flagged_fraud
FROM risk_scored_data
```


Results



FRAUD_TYPE

■ Account Takeover ■ Card Cloning ■ Friendly Fraud ■ Identity Theft ■ None ■ Phishing ■ Synthetic Identity





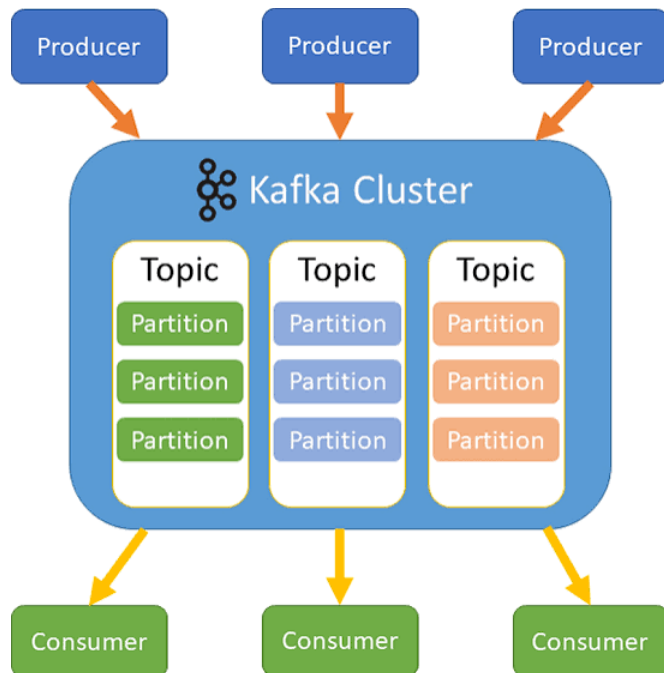
Phase 4

Kafka and Nifi



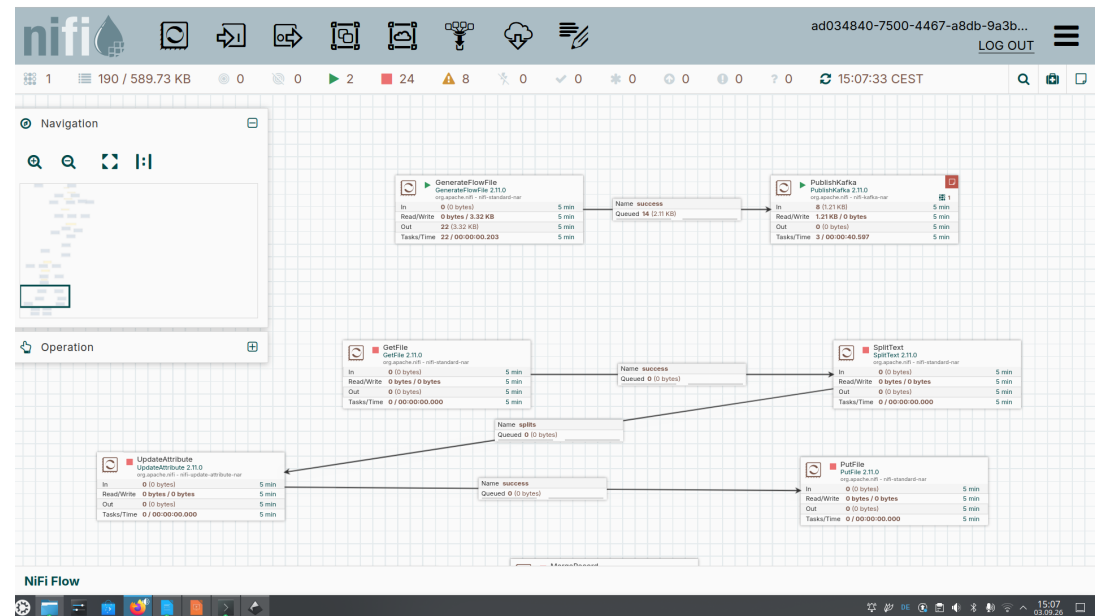
Streaming platform for building real time data pipeline and streaming action

Kafka acts as push-pull program



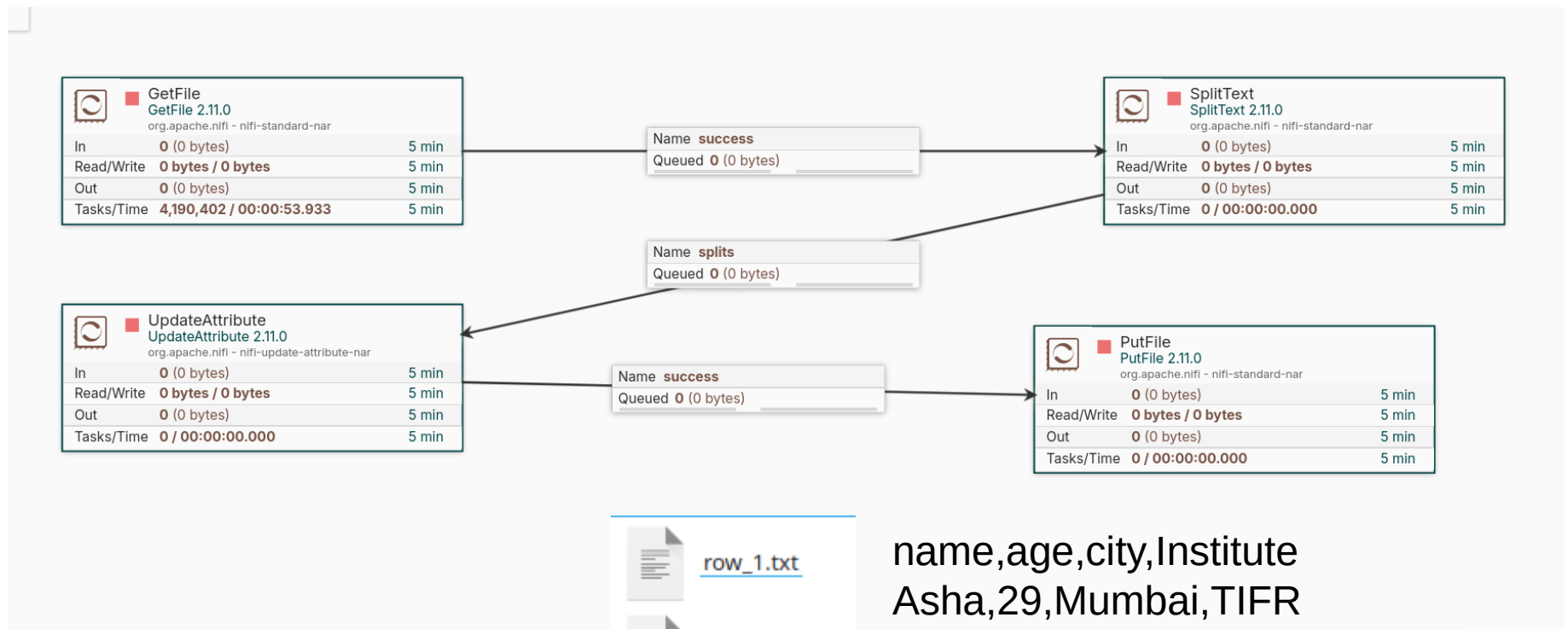
Launching platform of many processors, basically can do all the tasks what we learned before

Work-flow I program



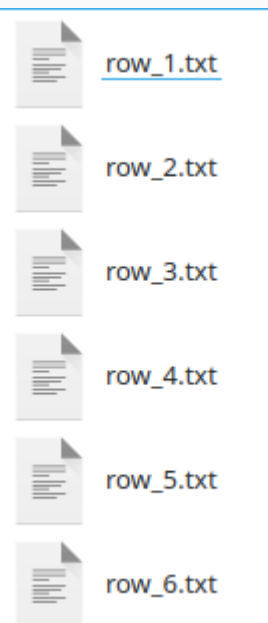
They run as separate services and each use its own network port.

Task 1: Build a NIFI flow that reads csv file from a local folder, splits each row into Individual flowfiles and write them to a destination folder with file name:row_.txt



Input (people.csv)

name,age,city,Institute
Asha,29,Mumbai,TIFR
Rahul,34,Pune,ICL
John,39,Munich,TUM
Alex,25,Madrid,IE
Tobi,50,London,IML
Ana,43,Lisbon,UL



name,age,city,Institute
Asha,29,Mumbai,TIFR

name,age,city,Institute
Rahul,34,Pune,ICL
name,age,city,Institute
John,39,Munich,TUM
name,age,city,Institute
Alex,25,Madrid,IE
name,age,city,Institute
Tobi,50,London,IML
name,age,city,Institute
Ana,43,Lisbon,UL

Task 4: Write a Python Kafka producer that reads rows from a CSV file and sends each row as a JSON message to a topic. Implement error handling and ensure at-least-once delivery using `acks='all'`.

1. **Start kafka service:** `bin/kafka-server-start.sh config/server.properties`
Check kafka is running: `bin/kafka-topics.sh`
`bin/kafka-console-producer.sh`
`bin/kafka-console-consumer.sh`
`bin/kafka-server-start.sh`
2. **Create topic :** `bin/kafka-topics.sh --bootstrap-server localhost:9092 --create --topic FinalCapstone --partitions 1 --replication-factor 1`
3. **Inspect the topic** `$ bin/kafka-topics.sh --bootstrap-server localhost:9092 --list`
4. **Producer Console:** python code --- Pushing msg to kafka
Reading csv file, sending msg to kafka, opening json file
Writing json file each csv row as json.
5. **Console Consumer** `$ bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic FinalCapstone --from beginning`

Pulling msg from kafka producer console.

Python code, input file out put file all are submitted.

Results

```
Row 433 sent successfully (partition=0, offset=3640)
Row 434 sent successfully (partition=0, offset=3641)
Row 435 sent successfully (partition=0, offset=3642)
Row 436 sent successfully (partition=0, offset=3643)
Row 437 sent successfully (partition=0, offset=3644)
Row 438 sent successfully (partition=0, offset=3645)
Row 439 sent successfully (partition=0, offset=3646)
Row 440 sent successfully (partition=0, offset=3647)
Row 441 sent successfully (partition=0, offset=3648)
Row 442 sent successfully (partition=0, offset=3649)
Row 443 sent successfully (partition=0, offset=3650)
Row 444 sent successfully (partition=0, offset=3651)
Row 445 sent successfully (partition=0, offset=3652)
Row 446 sent successfully (partition=0, offset=3653)
Row 447 sent successfully (partition=0, offset=3654)
Row 448 sent successfully (partition=0, offset=3655)
Row 449 sent successfully (partition=0, offset=3656)
Row 450 sent successfully (partition=0, offset=3657)
Row 451 sent successfully (partition=0, offset=3658)
Row 452 sent successfully (partition=0, offset=3659)
Row 453 sent successfully (partition=0, offset=3660)
Row 454 sent successfully (partition=0, offset=3661)
Row 455 sent successfully (partition=0, offset=3662)
Row 456 sent successfully (partition=0, offset=3663)
Row 457 sent successfully (partition=0, offset=3664)
Row 458 sent successfully (partition=0, offset=3665)
```

CSV processing completed.

JSONL file saved: nba_messages.jsonl

JSON file saved: nba_messages.json

(base) susmita@roy:~/Susmita/Projects/Data_Engg/Kafka/

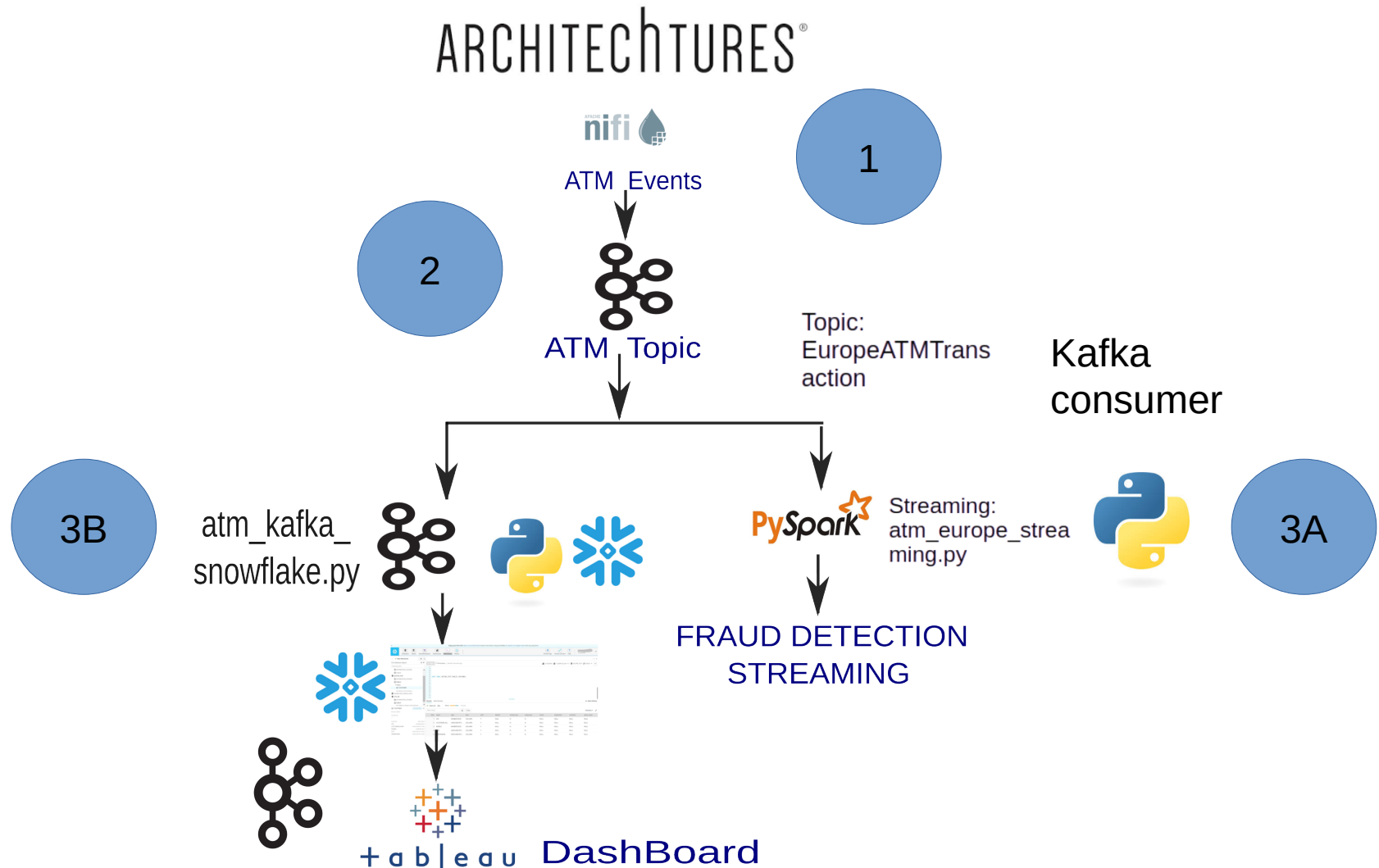
```
"Name": "Noah Vonleh", "Team": "Portland Trail Blazers", "Number": "21.0", "Position": "PF", "Age": "20.0", "Height": "6-9", "Weight": "240.0", "College": "Indiana", "Salary": "2637720.0"}
{"Name": "Trevor Booker", "Team": "Utah Jazz", "Number": "33.0", "Position": "PF", "Age": "28.0", "Height": "6-8", "Weight": "228.0", "College": "Clemson", "Salary": "4775000.0"}
{"Name": "Trey Burke", "Team": "Utah Jazz", "Number": "3.0", "Position": "PG", "Age": "23.0", "Height": "6-1", "Weight": "191.0", "College": "Michigan", "Salary": "2658240.0"}
{"Name": "Alec Burks", "Team": "Utah Jazz", "Number": "10.0", "Position": "SG", "Age": "24.0", "Height": "6-6", "Weight": "214.0", "College": "Colorado", "Salary": "9463484.0"}
{"Name": "Dante Exum", "Team": "Utah Jazz", "Number": "11.0", "Position": "PG", "Age": "20.0", "Height": "6-6", "Weight": "190.0", "College": "", "Salary": "3777720.0"}
{"Name": "Derrick Favors", "Team": "Utah Jazz", "Number": "15.0", "Position": "PF", "Age": "24.0", "Height": "6-1", "Weight": "265.0", "College": "Georgia Tech", "Salary": "12000000.0"}
{"Name": "Rudy Gobert", "Team": "Utah Jazz", "Number": "27.0", "Position": "C", "Age": "23.0", "Height": "7-1", "Weight": "245.0", "College": "", "Salary": "1175880.0"}
{"Name": "Gordon Hayward", "Team": "Utah Jazz", "Number": "20.0", "Position": "SF", "Age": "26.0", "Height": "6-8", "Weight": "226.0", "College": "Butler", "Salary": "15409570.0"}
{"Name": "Rodney Hood", "Team": "Utah Jazz", "Number": "5.0", "Position": "SG", "Age": "23.0", "Height": "6-8", "Weight": "206.0", "College": "Duke", "Salary": "1348440.0"}
{"Name": "Joe Ingles", "Team": "Utah Jazz", "Number": "2.0", "Position": "SF", "Age": "28.0", "Height": "6-8", "Weight": "226.0", "College": "", "Salary": "2050000.0"}
{"Name": "Chris Johnson", "Team": "Utah Jazz", "Number": "23.0", "Position": "SF", "Age": "26.0", "Height": "6-6", "Weight": "206.0", "College": "Dayton", "Salary": "981348.0"}
{"Name": "Trey Lyles", "Team": "Utah Jazz", "Number": "41.0", "Position": "PF", "Age": "20.0", "Height": "6-10", "Weight": "234.0", "College": "Kentucky", "Salary": "2239800.0"}
{"Name": "Shelvin Mack", "Team": "Utah Jazz", "Number": "8.0", "Position": "PG", "Age": "26.0", "Height": "6-3", "Weight": "203.0", "College": "Butler", "Salary": "2433333.0"}
{"Name": "Raul Neto", "Team": "Utah Jazz", "Number": "25.0", "Position": "PG", "Age": "24.0", "Height": "6-1", "Weight": "179.0", "College": "", "Salary": "900000.0"}
{"Name": "Tibor Pleiss", "Team": "Utah Jazz", "Number": "21.0", "Position": "C", "Age": "26.0", "Height": "7-3", "Weight": "256.0", "College": "", "Salary": "2900000.0"}
{"Name": "Jeff Withey", "Team": "Utah Jazz", "Number": "24.0", "Position": "C", "Age": "26.0", "Height": "7-0", "Weight": "231.0", "College": "Kansas", "Salary": "947276.0"}
{"Name": "", "Team": "", "Number": "", "Position": "", "Age": "", "Height": "", "Weight": "", "College": "", "Salary": ""}
```

CPprocessed a total of 3666 messages

(base) susmita@roy:~/Susmita/Projects/Data_Engg/Kafka/kafka_2.13-4.3.1\$

Real-Time ATM Transaction Monitoring in Germany

A German bank operates ATMs across Germany. A real-time streaming pipeline that generates ATM transactions And get information about Suspicious Withdrawn





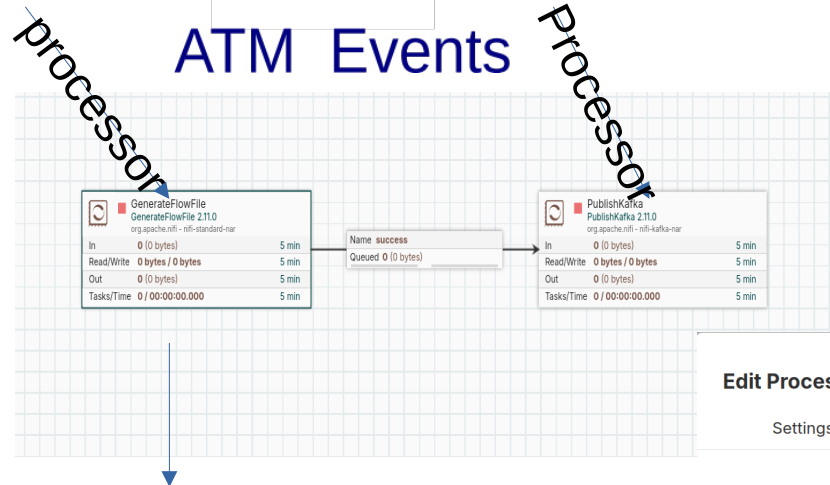
ATM Events

1

Kafka-consumer

```
---bin/kafka/console-  
consumer.sh/ --bootstrap-  
server localhost:9092 --topic  
EuropeATMTransaction
```

2



```
Custom text: {  
  "transaction_id": "TXN${  
    {random():mod(100000)}",  
  "atm_id": "ATM${random():mod(50)}",  
  "country": "Germany",  
  "city": "Berlin",  
  "amount": "${random():mod(5000)}",  
  "transaction_type": "Withdrawal"  
}
```



1. GenerateFlowFile

2. KafkaPublisher

```
kafka_2.13-4.3.1 : java — Konsole <2>  
File Edit View Bookmarks Plugins Settings Help  
{  
  "transaction_id": "TXN1016",  
  "atm_id": "ATM37",  
  "country": "Germany",  
  "city": "Berlin",  
  "amount": "1037",  
  "transaction_type": "Withdrawal"  
}  
{  
  "transaction_id": "TXN76418",  
  "atm_id": "ATM1",  
  "country": "Germany",  
  "city": "Berlin",  
  "amount": "1294",  
  "transaction_type": "Withdrawal"  
}  
{  
  "transaction_id": "TXN59474",  
  "atm_id": "ATM26",  
  "country": "Germany",  
  "city": "Berlin",  
  "amount": "2889",  
  "transaction_type": "Withdrawal"  
}  
{  
  "transaction_id": "TXN43186",  
  "atm_id": "ATM43",  
  "country": "Germany",  
  "city": "Berlin",  
  "amount": "2544",  
  "transaction_type": "Withdrawal"  
}
```




ATM Events

1

Kafka-consumer

```
---bin/kafka/console-  
consumer.sh/ --bootstrap-  
server localhost:9092 --topic  
EuropeATMTransaction
```

2

processor

GenerateFlowFile
GenerateFlowFile 2.11.0
org.apache.nifi - nifi-standard-nar

In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

processor

PublishKafka
PublishKafka 2.11.0
org.apache.nifi - nifi-kafka-nar

In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

Custom text: {
 "transaction_id":"TXN\${
random():mod(100000)}",
 "atm_id":"ATM\${random():mod(50)}",
 "country":"Germany",
 "city":"Berlin",
 "amount":"\${random():mod(5000)}",
 "transaction_type":"Withdrawal"
}



1. GenerateFlowFile

2. KafkaPublisher

Edit Processor | PublishKafka 2.11.0

Settings

Scheduling

Properties

Required field

Property	Value
Kafka Connection Service	Kafka3ConnectionService
Topic Name	EuropeATMTransaction
Failure Strategy	Route to Failure
Delivery Guarantee	Guarantee Replicated Delivery
Compression Type	none
Max Request Size	1 MB
Transactions Enabled	false

Stopped



atm_europe_streaming.py

3A



```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json, avg, count, sum as _sum
from pyspark.sql.types import StructType, StructField, StringType, DoubleType

# -----
# 1. Spark session
# -----
spark = (
    SparkSession.builder
        .appName("ATM_Europe_Streaming")
        .getOrCreate()
)

spark.sparkContext.setLogLevel("WARN")

# -----
# 2. Read raw stream from Kafka
# -----
raw_stream = (
    spark.readStream
        .format("kafka")
        .option("kafka.bootstrap.servers", "localhost:9092")
        .option("subscribe", "EuropeATMTransaction")
        .option("startingOffsets", "latest")
        .load()
)

# Kafka gives us key/value as bytes - the JSON payload is in "value"
raw_json = raw_stream.selectExpr("CAST(value AS STRING) AS json_value")

# -----
# 3. Define schema and parse JSON, cast amount to numeric
```

1. Read Structured raw stream from Kafka
2. Define schema and parse JSON,
3. Real-Time Values: Transactions, Withdrawal amount
4. Fraud Detection: flag withdrawals above 4000
FRAUD_THRESHOLD = 4000
fraud_alerts = withdrawals.filter(col("amount") > FRAUD_THRESHOLD).select(col("transaction_id"), col("country"), col("city"), col("amount"),)
fraud_query =
(fraud_alerts.writeStream .outputMode("append") # each new suspicious row, once .format("console") .option("truncate", False) .queryName("fraud_console") .start())
4. Keep the system Running

Results of Pyspark Streaming

Kafka_Nifi_Pyspark : java — Konsole

File Edit View Bookmarks Plugins Settings Help

```
-----+-----+-----+-----+
```

```
-----+
tch: 627
-----
```

```
-----+-----+-----+-----+
total_transactions|total_withdrawal_amount|avg_withdrawal_amount|
-----+-----+-----+-----+
28                |1563314.0                |2489.353503184713    |
-----+-----+-----+-----+
```

```
-----+
tch: 628
-----
```

```
-----+-----+-----+-----+
transaction_id|country|city  |amount|
-----+-----+-----+-----+
XN84348       |Germany|Berlin|4552.0|
-----+-----+-----+-----+
```

```
-----+
tch: 628
-----
```

```
-----+-----+-----+-----+
total_transactions|total_withdrawal_amount|avg_withdrawal_amount|
-----+-----+-----+-----+
29                |1567866.0                |2492.632750397456    |
-----+-----+-----+-----+
```

Integration of Snowflake – Getting UI Based Streaming Data



1. Use Kafka Consumer

2. Snowflake Connection

```
conn = snowflake.connector.connect(
    user='*****',
    password='*****',
    account='*****',
    warehouse='COMPUTE_WH',
    database='ATM_STREAMING_DB',
    schema='ATM_DATA',
    role='ACCOUNTADMIN'
)
```

cursor = conn.cursor()

print("Listening to ATM Kafka TOPIC...")

3. Read Kafka and then Insert Into
SnowFlake

4. Keep the system Running



3B



```
16 SELECT *
17 FROM ATM_STREAMING_DB.ATM_DATA.ATM_TRANSACTIONS;
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
```

```
CREATE OR REPLACE SECURE VIEW V_LIVE_ATM_MINUTE_ANALYTICS AS
SELECT
-- 1. Truncate the stream timestamp to the nearest minute
DATE_TRUNC('MINUTE', ALERT_TIMESTAMP) AS TRANSACTION_MINUTE,
COUNTRY,
TRANSACTION_TYPE,
-- 2. Live Student Exercise Aggregation
COUNT(TRANSACTION_ID) AS TOTAL_TRANSACTIONS,
SUM(AMOUNT) AS TOTAL_AMOUNT,
AVG(AMOUNT) AS AVG_AMOUNT,
-- 3. Live Fraud Counters (Count alerts above $4,000)
```

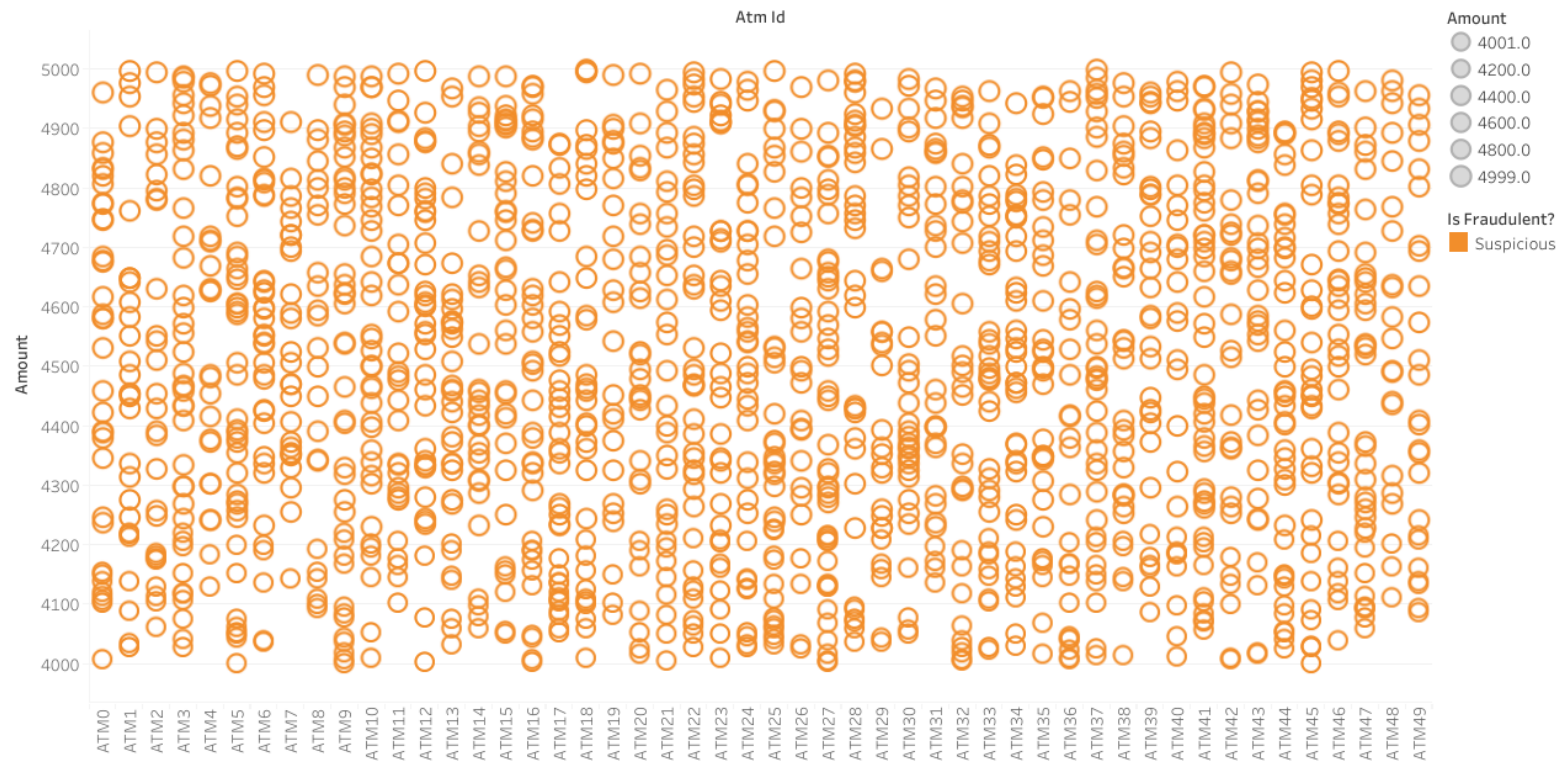
Results (just now)

	TRANSACTION_ID	ATM_ID	COUNTRY	CITY	AMOUNT	TRANSACTION_TYPE
1	TXN45044	ATM5	Germany	Berlin	4686	Withdrawal
2	TXN56887	ATM19	Germany	Berlin	2918	Withdrawal
3	TXN76913	ATM18	Germany	Berlin	4372	Withdrawal
4	TXN34865	ATM45	Germany	Berlin	3866	Withdrawal
5	TXN3721	ATM11	Germany	Berlin	1325	Withdrawal
6	TXN73406	ATM3	Germany	Berlin	1183	Withdrawal
7	TXN36939	ATM32	Germany	Berlin	3335	Withdrawal
8	TXN25344	ATM10	Germany	Berlin	4825	Withdrawal
9	TXN23674	ATM18	Germany	Berlin	41	Withdrawal
10	TXN25346	ATM2	Germany	Berlin	691	Withdrawal
11	TXN21406	ATM39	Germany	Berlin	2480	Withdrawal

Visualization



Is Fraudulent?





Thank you for your attention!

